

FREEZE/WIRE

Prove the event. Enforce the outcome.

Cryptographic counterparty compliance for Creditcoin: a real Ethereum mainnet Circle USDC `Blacklisted` fact becomes a Creditcoin eligibility state — only after Attestcoin proves inclusion and FreezeWire consumer checks bind the event.

LIVE · CC3 TESTNET 102031

ATTESTCOIN chainKey 3

TESTNET SCOPE

NOT MAINNET PRODUCTION

“The backend never told Creditcoin the address was blacklisted. The Attestcoin proof did.”

PUBLIC FRONTEND

frontend-bice-pi-49.vercel.app

PUBLIC BACKEND

freezewire-backend.onrender.com

GITHUB

github.com/CodewithJha/freeze-wire

LICENSE / STATUS

MIT · CC3 TESTNET LIVE

01 Executive summary

Creditcoin smart contracts cannot natively read Ethereum logs. When Circle marks an address `Blacklisted` on Ethereum mainnet USDC, CTC credit venues stay blind unless something bridges that fact. The naive bridge — a keeper calling `setRestricted` — makes a single key the compliance oracle.

FreezeWire inherits that Circle address-level flag into Creditcoin CC3 gated credit *without* a backend eligibility oracle. Path: real mainnet event → Attestcoin Merkle + continuity proof → BlockProver `0x0FD2` → FreezeWire consumer checks → `EligibilityLedger` → `GatedCreditLine`.



WHY ATTESTCOIN

Only cryptographic inclusion + continuity lets CC3 accept an Ethereum fact without trusting a worker narrative. Removal test: without `0x0FD2`, no path to `RESTRICTED`.

WHY CREDITCOIN

The venue that must refuse the draw is where the proof verifies. Enforcement (`GatedCreditLine`) lives beside `BlockProver` on CC3.

What is live today (verified)

Network	Creditcoin CC3 testnet · chainId <code>102031</code>
Attestcoin chainKey	<code>3</code> = Ethereum mainnet (not EVM chainId; Sepolia = <code>1</code>)
Public app	Vercel FE → Render worker → ETH / Proof Builder / CC3 (Mac-independent)
Demo outcome	Account <code>0xe05F...4A2A</code> = <code>RESTRICTED</code> ; <code>draw</code> reverts <code>Restricted()</code> · selector <code>0xcc08913</code>
Honesty	Featured account is already <code>RESTRICTED</code> from prior permissionless <code>submitProof</code> ; eligible actor shows “before”

No `setStatus` / `setRestricted`. Worker and UI are untrusted transport. Contracts are the eligibility authority.

02 The problem & the FreezeWire solution

The gap

External compliance facts (Circle’s address blacklist on Ethereum) and on-chain credit decisions on Creditcoin live on different machines of consensus. Bridging them with APIs, databases, admin setters, or manually maintained status lists creates a single point of failure: compromise, downtime, or censorship of that operator yields wrong freezes or continued lending to flagged addresses.

Desired property

An address becomes `RESTRICTED` only when the underlying Ethereum fact has been cryptographically proven and Creditcoin contracts independently verify that proof — not when a backend says so.

ORACLE TRAP

```
setRestricted(addr, bool) onlyOwner
```

Key = compliance oracle. LP trusts server narrative.

FREEZEWIRE

```
submitProof(calldata) permissionless
```

Invalid proofs revert on-chain. No setter exists.

Object of the gate

CEL freezes the instrument (e.g. issuer `Paused` → asset eligibility). **FreezeWire freezes the counterparty** after a proven Circle `Blacklisted` → address eligibility for new credit. Same Attestcoin spine class; different economic object. We do not claim overall superiority to CEL.

Attestcoin is not a label. It is the execution input.

03 Why Attestcoin is load-bearing

```
Ethereum event
→ source tx + receipt
→ Merkle inclusion + header continuity
→ BlockProver 0x0FD2 verifyAndEmit
→ FreezeWire consumer checks (status, emitter, topic, account, chainKey, window, txIndex)
→ EligibilityLedger state transition
→ GatedCreditLine enforcement
```

What `0x0FD2` proves: inclusion of the submitted encoded transaction under an attested header, plus continuity toward the attested tip.

What it does *not* prove: receipt success, Circle USDC emitter identity, event topic, or account. Those are FreezeWire consumer checks. Precompile success alone never writes `Restricted`.

Removal test: If Attestcoin / BlockProver `0x0FD2` is removed, FreezeWire cannot move an address to `RESTRICTED` — there is no owner setter, eligibility DB, or worker privilege that invents Circle's blacklist.

04 Live evidence — Ethereum → Attestcoin → CC3

Sources: `deployments/demo-evidence-public.json`, `demo-proof-public.json`, README, live API `/v1/status` (2026-09-13 re-check).

ETH source tx	<code>0xc9edfdbb...ff787</code> · Circle USDC <code>blacklist</code> → <code>Blacklisted</code>
Block / txIndex	<code>25,705,174</code> · recovered txIndex <code>18</code>
Account B	<code>0xe05F529f5284D75624eBa386CB716928c3b54A2A</code>

Emitter (immutable)	0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48
Proof Builder	chainKey 3 · siblings 9 · continuity roots 27
CC3 submitProof	0x07e30451...dfc45 · status success
statusOf(B)	RESTRICTED (artifact + live /v1/status source= chain)
draw as B	reverts · signature 0xccc08913 = Restricted()

Proof-to-enforcement timeline

1. Real Ethereum mainnet Circle USDC transaction exists (not Sepolia, not a mock blacklist).
2. Proof Builder returns Merkle + continuity for chainKey 3 , height 25705174 , txIndex 18 .
3. On-chain verifyAndEmit at BlockProver 0x...0FD2 .
4. Consumer: receiptStatus == 1 ; emitter == constructor-immutable USDC; topic0 ∈ {Blacklisted, UnBlacklisted}; account from topics[1] .
5. Replay key burned; ledger writes RESTRICTED.
6. GatedCreditLine.draw as B reverts; repay / unused withdraw remain allowed.

Demo honesty: B is already RESTRICTED from a prior permissionless submit. Actor A (0x6b0745...cD52) shows the eligible path. Default unset = ELIGIBLE is fail-open — not “proven clean.”

05 Live contract deployments

CONTRACT	ADDRESS	ROLE
BlacklistVerifier	0x6bf23829...47D869	0x0FD2 + consumer checks · no eligibility storage
EligibilityLedger	0xde64d503...840C9d	Only eligibility writer · permissionless submitProof
GatedCreditLine	0xB04fFca2...9Ed340	Reads statusOf · enforces draw/escrow gates
MockUSD	0x6943EB32...DdC5B3	Demo ERC-20 on CC3 — not Circle USDC

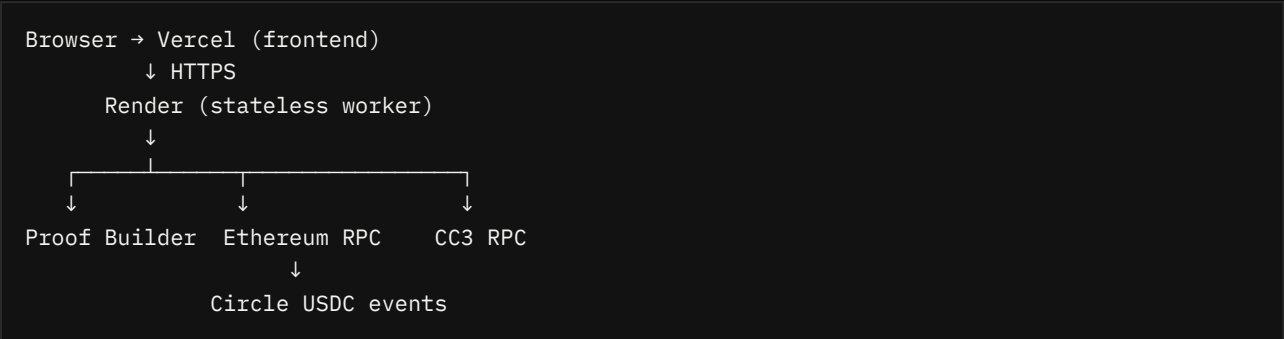
Deploy block 5479278 · LTV 5000 bps · proof window (0,0) unbounded (disclosed testnet residual) · BlockProver 0x...0FD2 .

GatedCreditLine does **not** call 0x0FD2 . No owner setStatus exists anywhere.

06 Deployed public application

TRY LIVE DEMO

Frontend	<code>frontend-bice-pi-49.vercel.app</code>
Backend	<code>freezewire-backend.onrender.com</code>
GitHub	<code>github.com/CodewithJha/freeze-wire</code>



Mac-independent: public path no longer requires localhost, a developer Mac, or Cloudflare Quick Tunnel. Verified: FE HTTP 200, `/v1/health` 200 with `cc3Rpc` / `ethRpc` / `proofBuilder.ok`, CORS allows the Vercel origin, `/v1/status/0xe05F...` returns `RESTRICTED` from chain.

API surface (transport only)

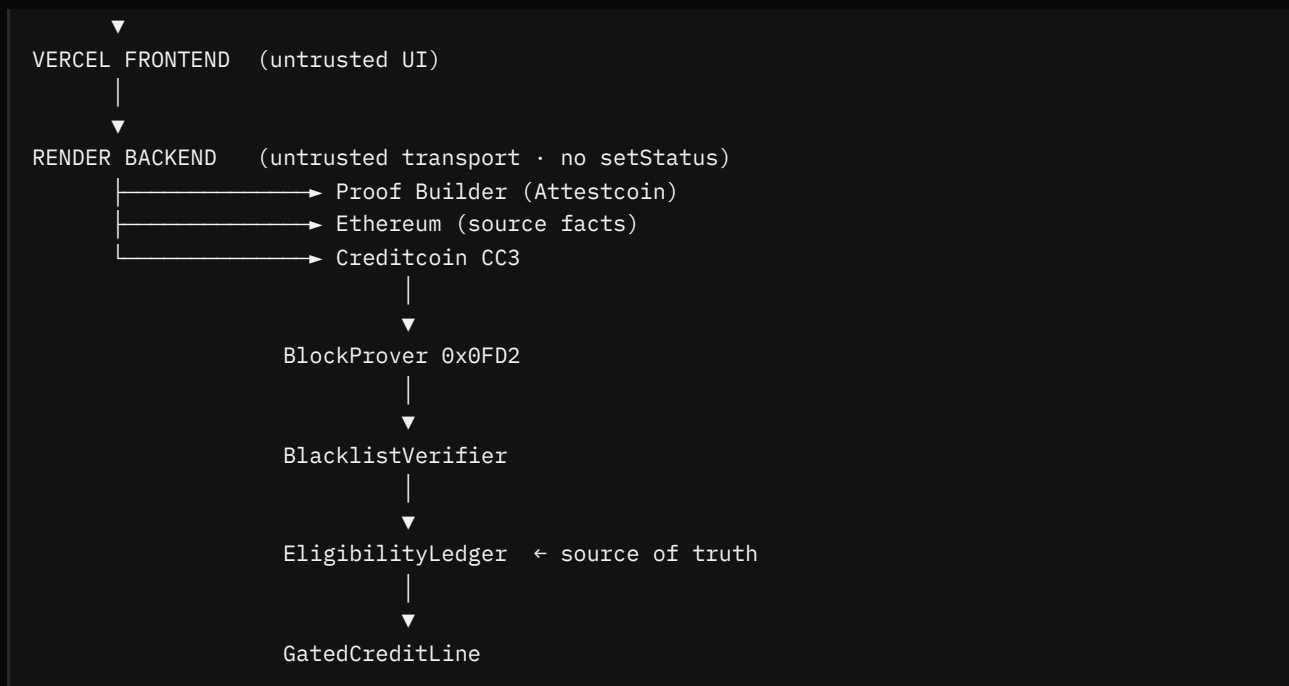
METHOD	PATH	NOTES
GET	<code>/v1/health</code>	Worker + RPC + Proof Builder health
GET	<code>/v1/evidence/demo</code>	Pinned public demo evidence
GET	<code>/v1/prove/{tx}</code>	Fetch Attestcoin proof bundle
GET	<code>/v1/status/{address}</code>	Read on-chain <code>statusOf</code>
POST	<code>/v1/relay</code>	Without key → <code>404 RELAY_DISABLED</code> + <code>submitProof</code> calldata

Relay intentionally disabled without `RELAY_PRIVATE_KEY` is a **security property**: the worker returns prepared calldata for wallet broadcast instead of holding broadcast privilege. Verified live against the demo tx.

Live UI: `frontend-bice-pi-49.vercel.app` — dark institutional evidence console · accent `#D6FF3F`. Screenshot asset retained in `submission-prep/whitepaper/assets/frontend-hero.png`.

07 End-to-end architecture

USER / WALLET



NOT AUTHORITY

Frontend, backend, relay, caches, RPC until verified on-chain.

AUTHORITY

On-chain eligibility + explorers. Wallet controls broadcast.

NO DB ORACLE

Stateless worker (ADR-0004). Eligibility is not a database row.

08 Smart contract design

Three-contract separation (ADR-0003) reduces attack surface and clarifies trust:

CONTRACT	OWNS	DOES NOT
BlacklistVerifier	0x0FD2 call, decode, bind events	Store eligibility
EligibilityLedger	statusOf, processed replay map, submitProof	Call credit math; owner setRestricted
GatedCreditLine	deposit/draw/repay/withdraw/escrow gates	Contain Attestcoin verification

Solidity `0.8.23`, Foundry, custom errors, constructor-immutable Circle USDC emitter (ADR-0016).

5-stage verification spine

- Inclusion** — `0x0FD2.verifyAndEmit`
- Status** — `receiptStatus == 1` or `SourceTxFailed`
- Emitter** — `log.address == immutable Circle USDC`
- Event** — `topic0 Blacklisted / UnBlacklisted`
- Account** — `topics[1]`; caller cannot redirect

09 Security model & consumer checks

After precompile success, FreezeWire still requires: receipt status, immutable emitter, event topic, account topic, expected chainKey, height window, Merkle-recovered txIndex, ordering, and replay. Failed verify / decode / status≠1 does not create Restricted.

BlockProver proves inclusion/continuity. Consumer validates receipt success and application semantics. This distinction is load-bearing for Attestcoin-depth scoring.

Compact attack matrix (from SECURITY_EVIDENCE)

THREAT	DEFENSE	RESULT
Impostor emitter	Constructor-immutable USDC	No bind
Wrong topic / Pause decoy	topic0 filter	Ignored
Failed receipt	status==1	SourceTxFailed
Wrong chainKey	expected==3	WrongChainKey
Forged caller txIndex	recover from Merkle isLeft	Ignored
Bad Merkle / continuity	0x0FD2	ProofRejected
Replay	processed[key]	QueryAlreadyProcessed
Backend invents Restricted	No setter API	404 / impossible
Restricted draw	GatedCreditLine	Restricted()

10 Replay protection · ADR-0017 Model A

```
key = keccak256(abi.encodePacked(chainKey, height, recoveredTxIndex))
```

- Caller-supplied txIndex is informational only; on-chain index is recovered from Merkle path bits (`TxIndex.sol`).
- **Model A:** after successful native verify *and* successful receipt scan (`status==1`), mark `processed[key]=true` . If no canonical USDC Blacklisted/UnBlacklisted log → emit `ProcessedWithoutFact` and **return** (do not revert). Eligibility unchanged. Retry → `QueryAlreadyProcessed` .
- Do **not** mark replay if verify failed, window/chainKey failed, decode failed, or `receiptStatus != 1` .
- Stale observations (older than account `lastPos`) are skipped, not whole-tx reverts.

Rationale: EVM reverts undo storage; `processed=true; revert NoMatchingEvent` cannot burn the key. Emitter is constructor-immutable, so Model B retries are useless gas.

11 Decoder, backend, frontend

Attestcoin decoder

FreezeWire vendors Gluwa `EvmV1Decoder` in-process. It walks the **original** log order (preserving `logIndex` for monotonic ordering), keeps only immutable-USDC logs with Blacklisted/UnBlacklisted `topic0`, and binds the indexed account from `topics[1]`. Decoy Transfer/Pause logs in the same receipt are ignored without failing the tx when a later match exists.

Backend (Render)

Node.js ≥20 · TypeScript · viem · zod · `@gluwa/usc-sdk`. Scripts: `npm ci` · `npm run build` · `npm start` · `npm test`. Stateless discovery + proof fetch + optional relay. Error sanitization, CORS, body-size limits, in-memory rate-limit pruning, bounded retries. **Not** the compliance authority.

Frontend (Vercel)

React 19 · Vite · TypeScript · Tailwind CSS 4 · viem · Three.js / React Three Fiber. Story spine: HOOK → EVENT → PROOF → CREDITCOIN → ACCESS. Commit state machine: PREPARED → WALLET SIGNATURE → BROADCAST → RECEIPT CONFIRMED → COMMITTED. UI does not claim success before chain confirmation; `RELAY_DISABLED` surfaces as NOT BROADCAST with calldata. Explicit LIVE / NOT_DEPLOYED / UNAVAILABLE / SIMULATION modes — production deploy does not enable simulation as the default path.

12 Credit access enforcement

ACCOUNT B · RESTRICTED

`0xe05F...4A2A`
statusOf = RESTRICTED
draw / protectedTransfer / escrow lock|release →
revert
repay / unused withdraw / deposit → allowed

ACCOUNT A · ELIGIBLE DEMO

`0x6b0745...cD52`
Documented mint → approve → deposit → draw →
repay → withdraw txs in public evidence JSON
(CC3 testnet).

The restriction is enforced on-chain, not rendered by the UI.

13 Testing & verification

Counts verified locally 2026-09-13 via `forge test`, `backend npm test`, `frontend npm test`, `frontend npm run build`. Do not use older draft “44/5” figures.

SUITE	RESULT	COMMAND
Foundry	95 passed , 0 failed, 1 skipped (96 total; skip = optional live Attestcoin)	<code>forge test</code>

SUITE	RESULT	COMMAND
Backend	46 passed , 0 failed	<code>cd backend && npm test</code>
Frontend Vitest	17 passed , 0 failed	<code>cd frontend && npm test</code>
Frontend build	PASS	<code>cd frontend && npm run build</code>

Coverage includes adversarial receipts, emitter/topic/status/window/chainKey, txIndex recovery, replay, Model A no-match burn, gated finance T-FIN-*, API no-setter / RELAY_DISABLED, honesty UI states.

14 Threat & trust model

TRUSTED

Ethereum consensus · Attestcoin/BlockProver ·
Creditcoin consensus · immutable Circle USDC
address · deployed contract logic

UNTRUSTED

Frontend · backend · relay · user input · caches ·
RPC responses until verified on-chain

THREAT	ATTACK	STOPPED BY
Fake proof	Junk Merkle/continuity	0x0FD2
Malicious backend	Lie about blacklist	No setter; on-chain checks
Malicious UI	False “committed”	Honesty states + explorers
Leaked relay key	Gas grief / spam submit	Still cannot forge valid Restricted without proof
Unauthorized draw	Restricted borrower	GatedCreditLine

No setStatus(). No centralized compliance oracle.

15 Why not a centralized oracle?

CENTRALIZED

Backend says blacklisted → contract trusts
backend.

FREEZEWIRE

Ethereum event → proof → consumer checks →
state → credit gate.

There is intentionally no `setStatus(address, RESTRICTED)`. A compromised worker cannot invent Restricted; an honest worker cannot skip `0x0FD2`.

16 Product direction & differentiation

Product concept: external compliance facts become programmable credit constraints.

Framed as potential extensions — not existing customers or revenue.

- Compliance-aware CTC lending / stablecoin credit lines
- Counterparty risk controls for institutional venues
- Automated inheritance of issuer control-plane events (blacklist / restore)

CEL freezes the instrument; FreezeWire freezes the counterparty — only after Attestcoin proves the Ethereum fact.

PEER CLASS	THEIR OBJECT	FREEZEWIRE DIFFERENCE
CEL	Instrument pause	Address blacklist → credit gate
Corolary	Lending history / score	Compliance flag, not reputation
Toxa / Deadswitch	Lock / collateral prove	Inherit Circle control-plane; deny credit

Not merely another lending app, oracle, bridge, blacklist UI, or Attestcoin tutorial — the combination is: real mainnet compliance event + proof + consumer semantics + counterparty eligibility + enforceable CTC credit restriction + permissionless submit.

17 Innovation (precise)

1. Real Ethereum Circle USDC compliance event
2. Attestcoin inclusion + continuity
3. Consumer-side semantic validation
4. Counterparty eligibility state
5. Credit enforcement with never-trap exits
6. Permissionless proof submission
7. Wallet-controlled execution when relay is disabled
8. No centralized compliance oracle

18 Reproducibility

Prerequisites: Node.js ≥20, npm, Foundry (`forge` / `cast` / `anvil`), Git, EVM wallet, CC3 testnet CTC for broadcasts, Ethereum + Proof Builder HTTPS access.

```
git clone https://github.com/CodewithJha/freeze-wire.git
cd freeze-wire
cp .env.example .env
# fill public RPCs / addresses only — never commit secrets
# optional: frontend/.env.local via documented sync scripts

# Contracts
```

```
forge fmt --check && forge build && forge test

# Backend
cd backend && npm ci && npm test && npm run build && npm start

# Frontend
cd frontend && npm ci && npm test && npm run build && npm run dev
```

Prefer the **live** Vercel + Render deployment for verification. Local worker is for development. Commands requiring private keys / funded wallets are optional and must use operator-held secrets — none appear in this document.

Optional live Proof Builder tests: `LIVE_ATTESTCOIN=1 npm test` in `backend/`.

19 Repository structure

```
freeze-wire/
├── contracts/           # Foundry · BlacklistVerifier, EligibilityLedger, GatedCreditLine
├── backend/             # Stateless TypeScript worker (Render)
├── frontend/           # React 19 demo (Vercel)
├── deployments/        # *-public.json evidence (registries may be gitignored)
├── docs/               # Specs, ADRs, Attestcoin, security, demo
├── config/             # Network templates
├── scripts/            # Ops helpers
├── submission-prep/    # Whitepaper + project packaging docs
├── foundry.toml
└── README.md
```

20 Deployment architecture & live verification

Frontend host	Vercel · <code>frontend-bice-pi-49.vercel.app</code>
Backend host	Render · <code>freezewire-backend.onrender.com</code>
External	Ethereum mainnet · Proof Builder · Creditcoin CC3 testnet
Not required	Developer Mac · localhost · Cloudflare Quick Tunnel

Verified live checklist (2026-09-13)

CHECK	RESULT
Frontend reachable	✓ HTTP 200
Backend <code>/v1/health</code>	✓ 200 · cc3Rpc/ethRpc/proofBuilder ok · chainId 102031 · chainKey 3
CORS for Vercel origin	✓ allow-origin matches FE

CHECK	RESULT
ETH source tx	✓ Etherscan 200
submitProof tx	✓ Blockscout 200
Contract addresses on explorer	✓ all four 200
statusOf(B) via API	✓ RESTRICTED · source=chain
POST /v1/relay without key	✓ RELAY_DISABLED + submitProof calldata
Production FE simulation default	✓ not the public path (opt-in env only)

Live on Creditcoin CC3 Testnet.

21 Known limitations

- **CC3 testnet only** — not Creditcoin mainnet production; no audit claim.
- **Proof window (0,0)** unbounded on live deploy — disclosed residual; production should bound freshness.
- Owner can still retune `setExpectedChainKey` / `setWindow` (ops residual; not emitter swap; not Attestcoin bypass).
- MockUSD is demo collateral — not Circle reserves on CTC.
- Wallet path when relay disabled; in-memory rate limits; Render cold starts; limited FE E2E vs Foundry depth.
- Demo account already RESTRICTED — two-account honesty (documented).

22 Roadmap (not shipped)

Now: CC3 testnet + Vercel/Render. **Next:** bounded windows / monitoring. **Later:** more facts/chains. **Out of scope:** eligibility DB, writability-as-authority, rotatable emitter.

23 Summary

FreezeWire is an evidence-backed Attestcoin application on Creditcoin CC3 testnet that turns a **real Ethereum mainnet Circle USDC blacklist** into an on-chain **counterparty eligibility** state and a real **credit draw rejection** — without a backend compliance oracle.

- Meaningful Attestcoin depth: `0x0FD2` vs consumer checks, removal test, Model A replay
- Live CC3 deployment with public addresses and explorer txs
- Actual state transition to RESTRICTED + `Restricted()` enforcement
- Strong trust boundaries: no `setStatus`; RELAY_DISABLED + calldata
- Mac-independent public stack: Vercel → Render → ETH / PB / CC3
- Verified tests: Foundry 95/1 skip · backend 46 · frontend 17 · build PASS

- Clear product differentiation vs instrument-freeze (CEL) and scoring (Corolary)
- Honest limitations disclosed (testnet, unbounded window, already-RESTRICTED demo)

Proof verified → state changed → credit gate enforced.

If Attestcoin is removed, the compliance transition disappears.

24 Links

GitHub	github.com/CodewithJha/freeze-wire
Frontend	frontend-bice-pi-49.vercel.app
Backend health	/v1/health
ETH source tx	Etherscan 0xc9ed...f787
CC3 submitProof	Blockscout 0x07e3...fc45
Attestcoin summary	docs/ATTESTCOIN_INTEGRATION_SUMMARY.md
Public evidence	demo-evidence-public.json
Proof Builder health	proof-gen-api.cc3-testnet.../health

Document generated from repository source of truth. No secrets. No mainnet-production claims. Accent #D6FF3F matches the public product UI.

“The backend never told Creditcoin the address was blacklisted. The Attestcoin proof did.”